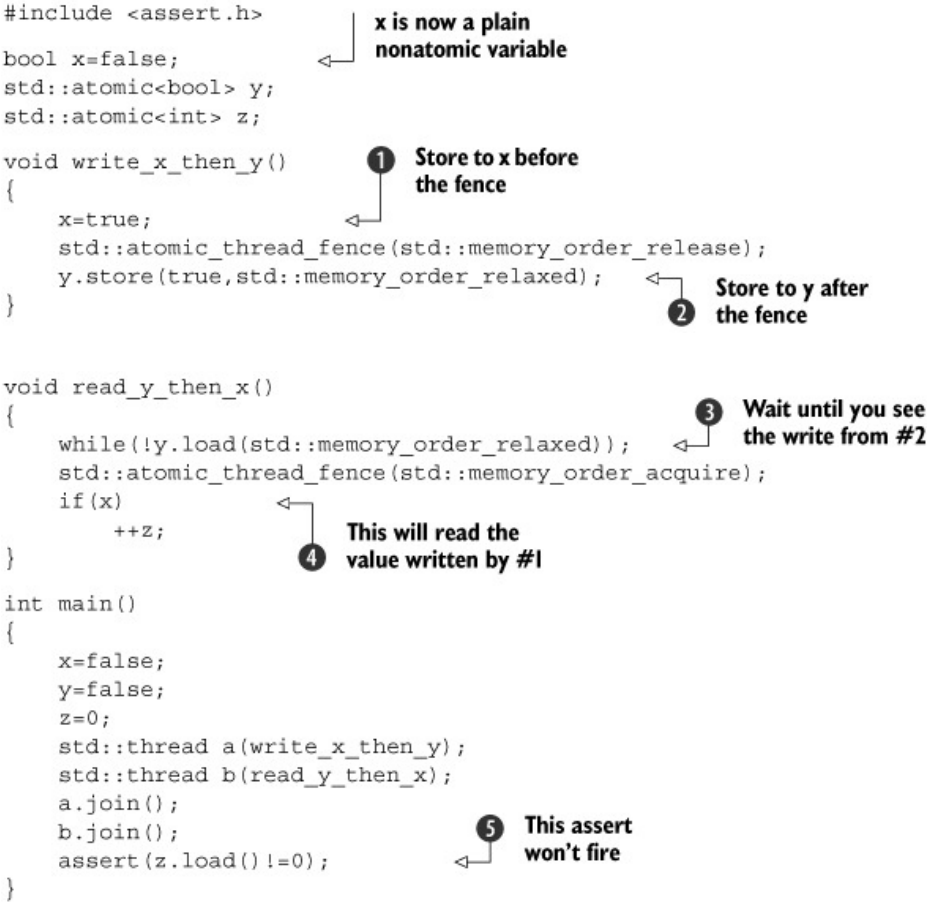
```
#include <atomic>
#include <thread>
#include <assert.h>

bool x=false;
std::atomic<bool> y;
std::atomic<int> z;

void write_x_then_y()
{
    x=true;
    std::atomic_thread_fence(std::memory_order_release);
    y.store(true,std::memory_order_relaxed);
}


void read_y_then_x()
{
    while(!y.load(std::memory_order_relaxed));
    std::atomic_thread_fence(std::memory_order_acquire);
    if(x)
        ++z;
}

int main()
{
    x=false;
    y=false;
    z=0;
    std::thread a(write_x_then_y);
    std::thread b(read_y_then_x);
    a.join();
    b.join();
    assert(z.load()!=0);
}
```



The fences still provide an enforced ordering of the store to `x` ❶ and the store to `y` ❷ and the load from `y` ❸ and the load from `x` ❹, and there's still a happens-before relationship between the store to `x` and the load from `x`, so the assert ❺ still won't fire. The store to ❷ and load from ❸`y` still have to be atomic; otherwise, there would be a data race on `y`, but the fences enforce an ordering on the operations on `x`, once the reading thread has seen the stored value of `y`. This enforced ordering means that there's no data race on `x`, even though it's modified by one thread and read by another.

It's not just fences that can order nonatomic operations. You saw the ordering effects back in listing 5.10 with a `memory_order_release`/`memory_order_consume` pair ordering nonatomic accesses to a dynamically allocated object, and many of the examples in this chapter could be rewritten with some of the `memory_order_relaxed` operations replaced with plain nonatomic operations instead.

Ordering of nonatomic operations through the use of atomic operations is where the sequenced-before part of happens-before becomes so important. If a nonatomic operation is sequenced-before an atomic operation, and that atomic operation happens-before an operation in another thread, the nonatomic operation also happens-before that operation in the other thread. This is where the ordering on the operations on `x` in listing 5.13 comes from and why the example in